树

5.1 树和二叉树的定义

- 树的定义（Tree）
- 树的基本术语
- 二叉树的定义（Binary Tree）

5.2 案例引入

- 数据压缩：哈夫曼编码
- 二叉树求解表达式

5.3 抽象数据类型定义ADT

5.4 二叉树的性质和存储结构

5.4.1 二叉树的性质

- 满二叉树
- 完全二叉树

性质1~5

5.4.2 二叉树的存储结构

注意：这里是 **二叉树** 的存储结构（普通树的话，链式指针的意义有所不同）

- （顺序存储结构 -- 基本不会用到）
- 链式存储结构
  - 二叉链表：数据域，左、右指针域（二叉树里是“左孩子”和“右孩子”）
  - 三叉链表：数据域，左孩子指针域、右孩子指针域、双亲节点指针域

```
//二叉树的二叉链表存储表示
typedef struct BiTNode{
    TElemType data;
    struct BiTNode *lchild, *rchild;
}BiTNode, *BiTree
```

5.5 遍历二叉树和线索二叉树

5.5.1 遍历二叉树

核心：线性化

- 先序遍历
- 中序遍历
- 后序遍历
- 层次遍历

```
//中序遍历的递归算法
void InOrderTraverse(BiTree T){
    if(T){
        InOrderTraverse(T->lchild);
        cout << T->data;
        InOrderTraverse(T->rchild);
    }
}
```

```
//中序遍历的非递归算法
void InOrderTraverse(BiTree T){
    InitStack(S);    //申请一个空栈
    BiTNode *p = T;           //用指针p指向根节点
    while( p || !StackEmpty(S)){
        if(p){       //p非空
            Push(S, p);
            p = p->lchild;
        }
        else{        //p为空
            Pop(S, q);         //出栈
            cout << q->data;   //访问根节点
            //先序遍历需要把“访问”这一行代码放到Push(S, p)之前的那一行
            p = q->rchild;     //遍历右子树
        }
    }
}
```

```
//后续遍历的非递归算法
void PostOrderTraverse(BiTree T){
    InitStack(S);
    BiTNode *p = T;
    BiTNode *q;
    BiTNode last = NULL;
```

```
    while(p || !StackEmpty(S)){
        if(p){
            Push(S, p);
            p = p->lchild;
        }
        else{
            GetTop(S, q);   //先看栈顶

            //如果右子树存在且没有被访问过，就先去右子树
            if(q->rchild && q->rchild != last){
                p = q->rchild;
            }
            else{
                Pop(S, q);          //否则说明可以访问根了
                cout << q->data;
                last = q;           //标记刚访问过
            }
        }
    }
}
```

由 二叉树的 **先序序列和中序序列** 或者 **后序序列和中序序列** 可以唯一的确定一颗二叉树。（先序序列和后序序列不行，因为无法确定左右子树两部分）

二叉树遍历算法 （递归算法）的应用：

- 创建二叉树的存储结构——二叉链表 （通常都是用**先序遍历**来创建）
- 复制二叉树（用先序遍历）
- 计算二叉树的深度
- 统计二叉树中节点的个数

接下来我一一展示代码：

```
//先序遍历创建二叉链表
void CreateBiTree(BiTree &T){     //C++的引用功能
    cin >> ch;
    if(ch == '#'){
        T == NULL;
    }
    else{
        T = new BiTNode;          //这里注意一下
        T->data = ch;
        CreateBiTree(T->lchild);
        CreateBiTree(T->rchild);
```

```
    }
}
```

```
//复制二叉树（先序遍历）
void Copy(BiTree T, BiTree &newT){
    if(T == NULL){
        newT == NULL;
    }
    else{
        newT = new BiTNode;
        newT->data = T->data;
        Copy(T->lchild, newT->lchild);
        Copy(T->rchild, newT->rchild);
    }
}
```

```
//计算二叉树的深度
int Depth(BiTree T){
    if(T == NULL){
        return 0;
    }
    else{
        int m = Depth(T->lchild);
        int n = Depth(T->rchild);
        if(m > n){
            return m + 1;
        }
        else{
            return n + 1;
        }
    }
}
```

```
//计算二叉树节点的个数
int NodeCount(BiTree T){
    if(T == NULL){
        return 0;
    }
    else{
        return NodeCount(T->lchild) + NodeCount(T->rchild) + 1;
    }
}
```

5.5.2 线索二叉树

- 基本概念
- 线索链表、线索、线索二叉树、线索化

$$LTag = \begin{cases} 0, & \text{lchild域指示结点的左孩子} \\ 1, & \text{lchild域指示结点的前驱} \end{cases}$$

$$RTag = \begin{cases} 0, & \text{rchild域指示结点的右孩子} \\ 1, & \text{rchild域指示结点的后继} \end{cases}$$

```
//二叉树的二叉线索存储表示
typedef struct BiThrNode{
    TElemType data;
    struct BiThrNode *lchild, *rchild;
    int LTag, RTag;
}BiThrNode, *BiThrTree;
```

在实际使用的过程中，最常使用的就是中序线索化。

- 算法5.7： 以节点p为根的子树中序线索化（每到一个节点，看p的左孩子、pre的右孩子）

```
void InThreading(BiThrTree p){
    if(p){
        InThreading(p->lchild);

        if(p->lchild){
            p->LTag = 0;
        }
        else{
            p->LTag = 1;
            p->lchild = pre;
        }

        if(pre->rchild){
            pre->RTag = 0;
        }
        else{
            pre->RTag = 1;
            pre->rchild = p;
        }
        pre = p;

        InThreading(p->rchild);
```

```
    }
}
```

- 算法5.8： 带头结点的二叉树中序线索化

```
void InOrderThreading(BiThrTree &Thrt, BiThrTree T){
//中序遍历二叉树T，并将其中序线索化，Thrt指向头结点
    Thrt = new BiThrNode;
    Thrt -> LTag = 0;
    Thrt -> RTag = 1;
    Thrt -> rchild = Thrt;
    if(T == NULL){
        Thrt -> lchild = NULL;
    }
    else{
        Thrt -> lchild = T;
        pre = Thrt;
        InThreading(T);
        pre -> rchild = Thrt;

        Thrt -> RTag = 1;
        Thrt -> rchild = pre;
    }
}
```

- 遍历线索二叉树（书上写了中序、先序、后序线索二叉树的结点查找方法，但是在此仅仅列出中序方法）
  在中序线索二叉树中查找
  - 查找 p 指针所指结点的**前驱**
    - 如果 p -> LTag == 1，那么 p -> lchild 即为结点的前驱
    - 如果 p -> LTag == 0，那么结点的前驱是 **左子树中最右下的结点**（遍历左子树时最后一个访问的结点）
  - 查找 p 指针所指结点的**后继**
    - 如果 p -> RTag == 1，那么 p -> rchild 即为结点的后继
    - 如果 p -> RTag == 0，那么结点的后继是 **右子树中最左下的结点**
- 算法5.9 遍历中序线索二叉树

```
void InOrderTraverse_Thr(BiThrTree T){
//T指向头结点，头结点的左链lchild指向根结点
//中序遍历二叉线索树T的非递归算法，对每个数据元素直接输出
     p = T -> lchild
     while(p != T){    //空树或遍历结束时，p == T
```

```
        while(p -> LTag == 0){      //沿着左孩子向下
            p = p->lchild;
        }
        cout << p->data;
        while(p -> RTag == 1 && p->rchild != T){   //沿着右线索访问后继结点
            p = p->rchild;
            cout << p->data;
        }
        p = p->rchild;      //转向 p 的右子树
    }
}
```

遍历线索二叉树的时间复杂度为 O(n)，空间复杂度为O(1)。

5.6 树和森林

5.6.1 树的存储结构

- 双亲表示法（每个结点有两个域，数据域 和 双亲指针域）
- 孩子表示法
- 孩子兄弟表示法（又称 “二叉树表示法” “二叉链表表示法”）（最普遍的树的表示方法）
  有三个域：数据域，两个指针域，分别指向该结点的 第一个孩子结点 和 下一个兄弟结点。

```
//树的二叉链表表示法
typedef struct CSNode{
    ElemType data;
    struct CSNode *firstchild, *nextsibling;
}CSNode, *CSTree;
```

孩子兄弟表示法可以把一个普通的树转换为二叉树。

5.6.2 森林与二叉树的转换

- 森林 -> 二叉树（类似上面的“孩子兄弟表示法，把右面的一棵树看做兄弟）
- 二叉树 -> 森林

5.6.3 树和森林的遍历

- 树的遍历
  - 先根遍历树
  - 后根遍历树
- 森林的遍历
  - 先序遍历

- 中序遍历

5.7 哈夫曼树及其应用

5.7.1 基本概念

路径，路径长度，树的路径长度，**权**，结点的带权路径长度，树的带权路径长度（**WPL**），哈夫曼树（又称最优树）。

5.7.2 哈夫曼树的构造算法

如果有 n 个叶子结点，则设置一个长度 2n-1 的一维数组来储存。（因为没有度为1的结点）
每个结点有：weight parent lchild rchild 四个域

```
//哈夫曼树的存储表示
typedef struct{
    int weight;                     //结点的权值
    int parent, lchild, rchild;     //结点的双亲、左孩子、右孩子的数组位置下标
}HTNode, *HuffmanTree;
```

算法5.10 构造哈夫曼树

```
void CreateHuffmanTree(HuffmanTree &HT, int n){
    if(n <= 1){
        return;
    }
    int m = 2*n -1;
    HT = new HTNode[m+1];   //0号单元弃用，所以总数加一

    //进行初始化
    for(int i=1; i<=m; i++){
        HT[i].parent = 0;
        HT[i].lchild = 0;
        HT[i].rchild = 0;
    }
    for(int i=1; i<=n; i++){
        cin >> HT[i].weight;
    }

    //开始创建哈夫曼树
    for(int i=n+1; i<=m+1; i++){
        Search(HT, i-1, s1, s2);
        //在HT[k](1<=k<=i-1)中选择两个双亲域为0，并且权值最小的结点s1和s2
        HT[s1].parent = i;
        HT[s2].parent = i;
```

```
        HT[i].lchild = s1;
        HT[i].rchild = s2;
        HT[i].weight = HT[s1].weight + HT[s2].weight;
    }
}
```

5.7.3 哈夫曼编码

- 前缀编码：任何一个编码都不是其他任何编码的前缀。
- 哈夫曼编码：哈夫曼树，每一个叶子结点的路径（左0，右1）对应一个编码
- 哈夫曼编码的两条性质
- 算法5.11 根据哈夫曼树求解哈夫曼编码

```
void CreateHuffmanCode(HuffmanTree HT, HummanCode &HC, int n){
//n代表的是叶子结点（要编码的东西）的个数
    HC = new char **[n+1];  //第0位弃用
    cd = new char[n];     //分配临时存放每个字符编码的动态数组空间
    cd[n-1] = '\0';
    for(int i=1; i<=n; i++){
        c = i;
        f = HT[i].parent;
        start = n-1;
        while(f != 0){
            start--;
            if(HT[i].lchild == c){
                cd[start] = '0';
            }
            else{
                cd[start] = '1';
            }
            c = f;
            f = HT[f].parent;
        }

        HC[i] = new char[n-start];
        strcpy(HC[i], &cd[start]);
    }
    delete cd;
}
```